Notas de estudo: Aceleradores de Processamento

(textos gerados por IA com base nos tópicos da aula e compilados/corrigidos/melhorados por João L. Sobral)
18-10-2026

---

1. O que são aceleradores de processamento?

Num modelo de computação heterogénea, a CPU atua como a unidade de controlo geral, gerindo a lógica sequencial e o sistema operativo, enquanto delega blocos de código matematicamente intensivos (*hotspots*) para hardware especializado conhecido como acelerador de processamento:

- **GPUs (Graphics Processing Units):** Arquiteturas massivamente paralelas compostas por milhares de núcleos pequenos, ideais para cálculo científico e inteligência artificial.
- **TPUs / NPUs:** Circuitos integrados de aplicação específica (ASICs) otimizados exclusivamente para operações de álgebra linear e redes neuronais profunda.
- **FPGAs:** Hardware reprogramável ao nível dos circuitos elétricos, oferecendo máxima eficiência para algoritmos customizados de baixa latência.

2. Processamento vetorial vs. matricial

Ambos são baseados em paralelismo de dados, mas manifesta-se em diferentes geometrias dependendo dos requisitos do algoritmo:

Característica	Processamento Vetorial	Processamento Matricial
Dimensão	Unidimensional (1D)	Bidimensional (2D) ou superior
Operando Alvo	Vetores (Linhas de dados / Arrays)	Matrizes de dados (ou Blocos)
Operação Comum	Soma de vetores, produto escalar	Multiplicação de matrizes
Foco de Aplicação	Simulações físicas, processamento de sinal	Treino/Inferência de Grandes Modelos de Linguagem (LLMs)

3. Hardware para suporte a processamento vetorial e SIMD

Os processadores modernos incluem hardware específico para processamento vetorial (e matricial nas gerações mais recentes), designadamente incluem instruções do tipo SIMD. SIMD significa *Single Instruction, Multiple Data* (uma instrução, múltiplos dados) e permite que uma única instrução de hardware opere sobre múltiplos dados em paralelo. Isto é suportado por:

- **Registos alargados (*vector registers*):** Em vez de guardarem um único número de 32 ou 64 bits, os registos guardam vários blocos de 32 ou 64 bits. Por exemplo, um registo de **512 bits** (como no AVX-512) pode armazenar, simultaneamente, dezasseis números de 32 bits (e.g,. IEEE FP32 *Single*

Precision Float-Point Number), ou oito números de 64 bits (e.g., , IEEE FP64, Double Precision Float-Point Number).

- **ALUs vetoriais:** Unidades logicas e aritméticas de processamento segmentadas que aplicam a mesma operação aritmética (ex: ADD) a todos os segmentos do registo vetorial ao mesmo tempo.
- **Leituras/escritas de vetores em memória:** leitura e escrita de vetores (por exemplo, 512 bits) em memória. As PUs mais modernas suportam vetores que não estejam armazenados em endereços contínuos de memória, mas deverão ter um espaçamento fixo, fazendo assim a recolha (*gather*) ou a dispersão (*scather*) na leitura/escrita.

As arquiteturas mais avançadas permitem flexibilidade dinâmica, ao contrário dos registos rígidos tradicionais, incluem registos polimórficos que permitem:

- **Flexibilidade dimensional:** Capacidade do hardware interpretar o banco de registos como vetores 1D longos ou blocos 2D conforme a necessidade.
- **Agnosticismo de comprimento (*Length-Agnostic*):** Em arquiteturas modernas, como o **ARM SVE**, o tamanho real do vetor de hardware (VLEN) não é fixado no código binário. O mesmo código corre num chip com registos de 128 bits ou de 2048 bits sem precisar de ser recompilado

3º Desafio: Desenvolver código [eficiente] para aceleradores

> ***Principais sub-desafios:***
> *1) **D1 - Paralelismo:** explorar paralelismo garantindo que não há dependências de dados (para explorar paralelismo ao nível dos dados, e.g., maps) e reduzir o impacto das dependências de controlo (e.g., garantir a ausência de desvios condicionais usando máscaras de predicação)*
> *2) **D2 - Gestão dos dados**: gestão explícita da memória (Implementar movimentação explícita de memória CPU <-> Acelerador); e garantir a localidade espacial dos dados (i.é., alinhamento de dados na memória).*
> *3) **Precisão numérica** – a maior parte do hardware atual é muito mais rápido a processar números com uma precisão mais reduzida. O principal desafio é baixar a precisão sem afetar significativamente a qualidade do resultado final*

Programar para aceleradores introduz vários desafios complexos que não existem na programação puramente sequencial (CPU). Alguns dos desafios mais importantes são:

- **Gestão de Memória Heterogénea:** Em plataformas aceleradoras com memória dedicada é necessário mover explicitamente os dados da memória RAM do sistema para a memória interna do acelerador (VRAM/HBM) através de barramentos como o PCIe, o que gera gargalos.
- **Ausência de desvios condicionais diretos:** Estruturas como if/else quebram o fluxo paralelo. O programador precisa utilizar Máscaras de Predicação (*Vector Masking*), onde todas as ramificações são calculadas e os resultados indesejados são filtrados por uma máscara de bits.

- **Alinhamento de dados e *strides*:** Para o hardware ler a memória de forma eficiente, os dados devem estar dispostos de forma consecutiva (coalescida). Padrões dispersos exigem instruções caras de *Gather*/*Scatter* (recolha e dispersão)

A **precisão numérica** surge como um dos maiores desafios específicos na utilização de aceleradores de processamento.. A escolha da precisão impacta diretamente o **desempenho**, o **consumo de largura de banda** de memória e a **fidelidade** dos resultados computacionais:

- **Gargalo de Largura de Banda (*Bandwidth Bottleneck*):** Mover dados entre a CPU/RAM e o acelerador é lento. Ao reduzir a precisão de **64 bits (FP64)** para **16 bits (FP16)**, transferimos 4 vezes menos dados, aliviando o barramento.
- **Taxa de Processamento (*Throughput*):** O hardware dos aceleradores modernos é desenhado para paralelizar operações mais simples. Muitas GPUs e TPUs conseguem processar o dobro ou o quádruplo de operações por segundo se utilizarem formatos de menor precisão.
- **Instabilidade Numérica:** Formatos menores têm menor alcance dinâmico, o que pode originar ***Underflow*** (números muito pequenos tornam-se zero, perdendo informação essencial) ou ***Overflow*** (números muito grandes excedem o limite do formato e transformam-se em 'NaN' ou infinito).

Para mitigar estes desafios de precisão, utilizam-se duas estratégias fundamentais:

- **Precisão Mista (*Mixed Precision*):** O acelerador executa as operações matemáticas mais pesadas (multiplicações de matrizes) em baixa precisão (**FP16/BF16**), mas acumula os resultados intermédios numa precisão maior (**FP32**). Isto garante velocidade sem perder a estabilidade dos dados.
- **Quantização (Quantization):** Processo de converter um modelo treinado em FP32 para números inteiros (como **INT8**). Isto reduz o tamanho dos modelos de IA para que caibam em telemóveis ou aceleradores de baixo consumo, sacrificando apenas uma fração mínima de precisão.